Akademia Techniczno-Artystyczna
Wydział Wrocławska Akademia Biznesu

Edge Detection Student Project

Image processing methods
Kamil Musiał, PhD, Eng.
Academic year 2026/2027

1 Goal and project files

Goal: understand how smoothing, noise and Canny thresholds change detected edges. You receive a complete program. Change only the marked parameters, run it and explain the saved results.

Task: complete seven controlled runs on a photograph and a synthetic scene. Compare relevant edges, unwanted responses and the synthetic boundary rates.

File or folder	Purpose
project.py	Ready program; edit the PARAMETERS block.
run_experiments.py	Optional automatic execution of all seven runs.
data/camera.png	Local photograph, 512 × 512 pixels.
requirements.txt	Three external libraries to install.
results/	Separate folder for every completed run.
DATA_SOURCES.txt	Image credit and technical sources.

What each library does

OpenCV (import cv2) reads images, blurs them and calculates Sobel, Laplacian and Canny results. NumPy (import numpy) stores pixel arrays, creates repeatable noise and calculates magnitude, angle and summary values. Matplotlib creates labelled comparison figures. Its Agg backend saves images without opening a window.

Built-in Python modules need no installation: pathlib handles file paths; csv writes measurement tables; json records settings; math calculates the blur kernel size; re checks safe run names. The optional runner imports project to change parameters before calling main().

2 Setup and first run

Use Python 3.11 or 3.12. Install Python from python.org if needed. On Windows, enable “Add Python to PATH”. Extract the entire supplied ZIP into a normal folder. Do not run inside the ZIP.

Windows open Command Prompt in the project folder

```
py -3.12 -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
python project.py
```

If Python 3.11 is installed, replace py -3.12 with py -3.11. Open the folder in File Explorer, type cmd in the address bar and press Enter to open Command Prompt there.

macOS or Linux open Terminal in the project folder

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python project.py
```

1. Open project.py in a plain-text editor, such as IDLE, VS Code or another code editor. Do not use Word.
2. Run the unchanged baseline once. Open results/baseline/photo_comparison.png and synthetic_comparison.png.
3. For each next run, restore the baseline values. Change RUN_NAME and exactly one listed parameter. Save the file, then run python project.py again.
4. Keep names in quotes and numbers without quotes. Use a decimal point, for example 1.2. An existing result folder is preserved; use a new name to repeat a run.

Optional: python run_experiments.py creates all seven prescribed runs. It skips existing run folders. Inspect each result and its settings even when you use this option.

3 Parameters to change

Find the PARAMETERS block near the top of project.py. The baseline values below are also the starting point for every controlled comparison.

Parameter	Default and allowed values	Meaning
RUN_NAME	"baseline"; letters, digits, _ or -	Unique result folder name.
GAUSSIAN_SIGMA	1.2; 0 to 5	Gaussian spread in pixels. 0 disables blur.
NOISE_STD	10.0; 0 to 60	Added noise in grey levels. 0 adds none.
LOW_THRESHOLD	40.0; 0 ≤ low < high	Allows weak edge candidates.
HIGH_THRESHOLD	100.0; low < high ≤ 1443	Creates strong edge seeds.
PHOTO_FILENAME	"camera.png"; a name in data/	Keep the supplied input fixed.
SEED	42; whole number 0-4294967295	Keep fixed for repeatable noise.
EDGE_TOLERANCE	2; whole number 0 to 5	Synthetic matching distance in pixels; keep fixed.

Keep these implementation choices fixed: 3 × 3 Sobel/Canny derivatives, L2gradient=True, Laplacian kernel size 3 and sign-change minimum range 20. They are recorded in settings.json.

The blur kernel covers about three sigma on each side: size = 2 × ceil(3 × sigma) + 1. Sigma 1.2 uses 9 × 9; sigma 3 uses 19 × 19. The code uses reflected borders.

Thresholds act on derivative strength, not input brightness. They can exceed 255. A display-normalised Sobel preview has a different scale and must not be used as a threshold lookup table.

4 Seven controlled runs

For each row, start from the baseline. Change only the parameter shown, plus RUN_NAME. Keep the image, seed and tolerance fixed. This makes the cause of each difference clear.

RUN_NAME	Change from baseline	Observation target
baseline	None: sigma=1.2, noise=10, low=40, high=100	Locate the person/tripod and the weak synthetic lower rectangle.
no_smoothing	GAUSSIAN_SIGMA = 0.0	Does background noise become unwanted edges?
strong_smoothing	GAUSSIAN_SIGMA = 3.0	Which fine photo details disappear?
low_threshold	LOW_THRESHOLD = 10.0	Which connected weak photo structures become longer?
high_threshold	HIGH_THRESHOLD = 180.0	Which photo structures lose strong seeds?
no_noise	NOISE_STD = 0.0	Which responses are stable without added noise?
high_noise	NOISE_STD = 30.0	Compare false responses and missed weak boundaries.

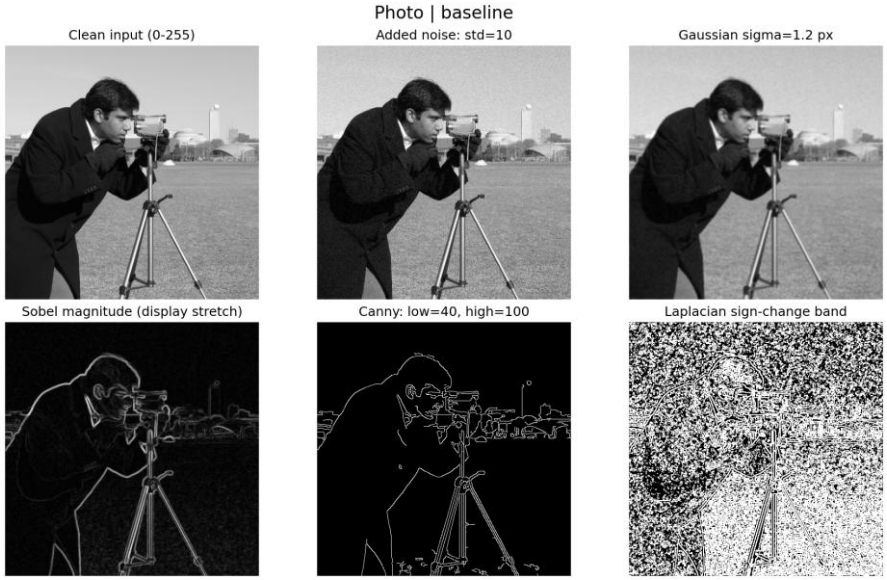
After each run, open both comparison images. Record photo edge count, synthetic edge count, precision and recall from measurements.csv. Save the exact settings, not only the run name.

Expected patterns are not universal rules. The synthetic scene can remain unchanged after a threshold change even when the photograph changes. Explain this result instead of assuming the program failed.

Baseline check with the supplied image and verified library versions: photo edge pixels 8,599; synthetic edge pixels 1,127; synthetic precision 1.000; recall about 0.745. Small version-dependent numerical differences are possible. Compare the actual saved images.

5 Read the outputs

Open results/RUN_NAME. photo_comparison.png and synthetic_comparison.png show clean input, noisy input, smoothed input, Sobel magnitude preview, Canny edges and a Laplacian candidate band or known reference. White Canny pixels are selected edges.



Baseline output: the same image is shown at successive processing steps.

Individual PNG files let you inspect detail. *_raw_arrays.npz preserves signed derivatives, magnitude, angle in radians and Laplacian. These arrays are optional; the report can use the PNG and CSV files.

settings.json records parameter values, versions and kernel size. measurements.csv contains counts and density; the synthetic row also contains boundary precision, recall and F1. Blank photo rates mean no boundary reference exists.

Processing order: add fixed-seed noise → explicit Gaussian blur → signed Sobel and Laplacian → Canny thinning and hysteresis. The Laplacian sign-change band is a separate teaching result, not an internal Canny stage.

6 Questions and files to submit

Answer with evidence from your runs

1. Compare baseline and no_smoothing. Identify one real photo structure and one noisy background area. Why does the edge count rise sharply without smoothing?

2. Compare baseline and strong_smoothing. Name one preserved large structure and one lost small detail. Explain the noise-detail tradeoff.

3. Compare the two threshold runs with baseline. Which parameter changed? Give one visible effect or explain why a chosen region stayed the same.

4. Compare no_noise, baseline and high_noise on the synthetic image. Report precision and recall. Explain why recall can rise while precision falls.

5. Compare synthetic_canny.png with synthetic_reference.png. Locate the weak lower rectangle and describe which boundary parts are missing. Does a clean-looking map guarantee complete detection?

6. Explain why Gx and Gy use floating-point storage. What information would unsigned 8-bit derivatives lose?

7. Explain why photo edge density cannot be interpreted as detection accuracy. State what extra reference data would be needed.

Submit a small organised report

Submit one PDF or Word report containing your seven-row settings-and-measurements table, answers to the questions, and a final choice of photo settings with a reason.

Include the baseline photo and synthetic comparisons, the no_smoothing synthetic comparison, the strong_smoothing photo comparison and the high_noise synthetic comparison. Add one crop that supports your final choice. Label every figure with its run name.

Also submit project.py with your final chosen parameters and the settings.json and measurements.csv from all seven runs. The report must state the two-pixel tolerance and distinguish photo observations from synthetic boundary measurements.

7 Help glossary and limitations

Problem	Action
No module named cv2 / numpy / matplotlib	Activate the virtual environment and run python -m pip install -r requirements.txt.
No files changed: folder already exists	Choose a new RUN_NAME. Existing results are preserved.
Cannot read camera.png	Extract all files. Keep data/ next to project.py.
SyntaxError after editing	Restore quotation marks and decimal points. Edit only the parameter block.
No window opens	Expected: open the saved PNG files in results/.
Very few edges	Check thresholds and blur. A sparse result can be valid; explain what is missing.

Glossary: gradient = strength and direction of local change; sigma = blur spread; non-maximum suppression = keep local ridge peaks; hysteresis = retain weak candidates linked to strong ones; precision = predicted boundary pixels near truth; recall = reference boundary pixels found nearby.

Limitations: synthetic truth is a one-pixel inside boundary, with many-to-many matching within two pixels. It is not a one-to-one benchmark. No truth is available for the photograph. The Laplacian mask is a sign-change band. Added Gaussian noise is only one noise model. Settings do not transfer unchanged to all image scales or contrasts.

Photo credit: Camera by Lav Varshney, CC0, via scikit-image. Technical sources: docs.opencv.org/4.x/d5/d0f/tutorial_py_gradients.html; docs.opencv.org/4.x/da/d22/tutorial_py_canny.html; docs.opencv.org/4.x/d5/db5/tutorial_laplace_operator.html; docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html. Image provenance and full URLs are in DATA_SOURCES.txt.